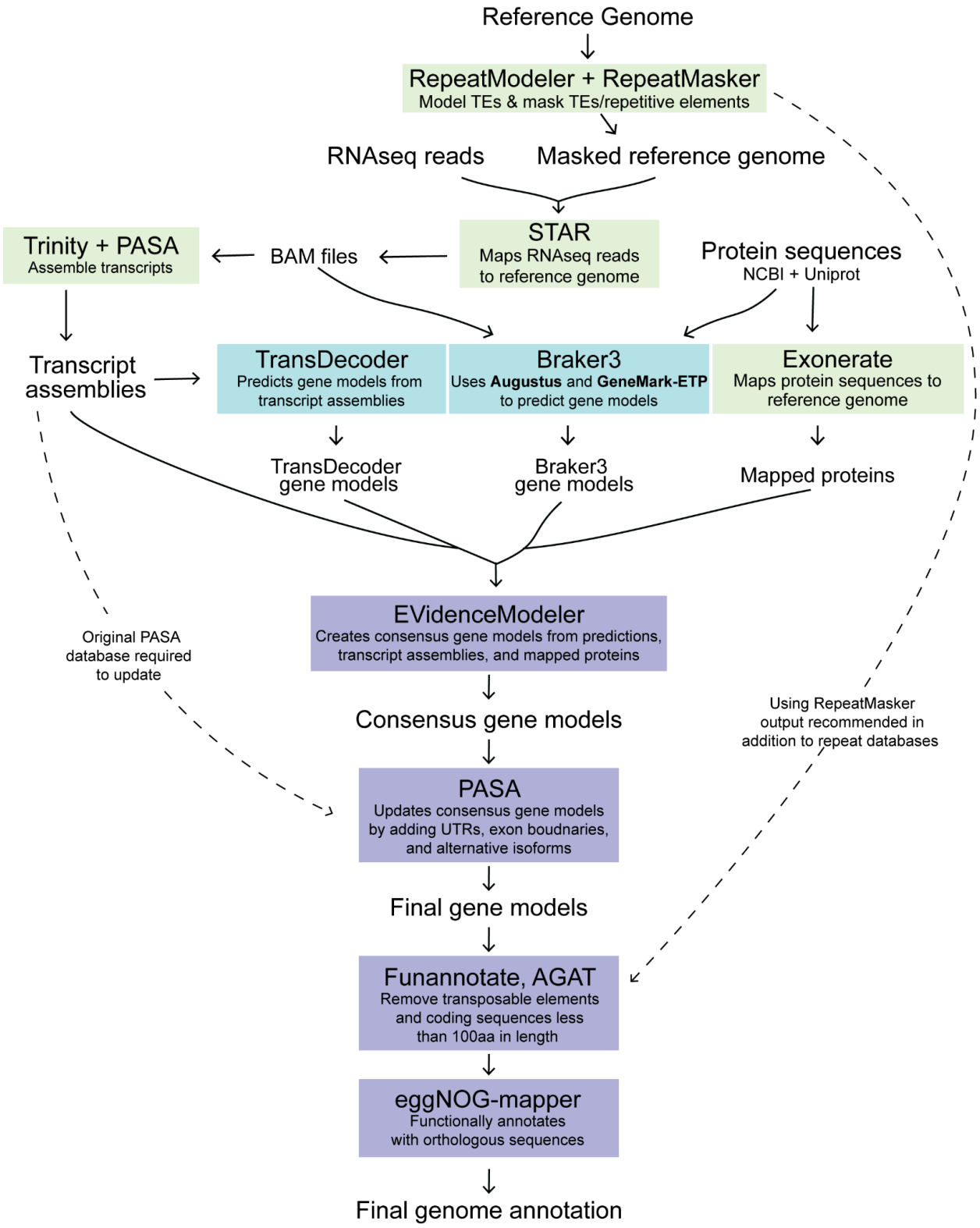
BRAKER3 + Evidence Modeler pipeline - Ayse Tenger-Trolander
Adapted from Nicholas Van Kuren's annotation pipelines



Directory Structure

First, let's set up our directory structure. Within some working directory (e.g. annotation/), set up the following directories. The example commands laid out in this document assume this structure, and all scripts are executed from the top level. Copy your raw RNA-seq data to transcript_data/fastqs/, your reference genome assembly to reference/, and all scripts to scripts/:

```
abinitio_data/
        braker/
protein_data/
        fastas/
        uniprot_results/
        refseq_results/
reference/
scripts/
transcript_data/
        fastqs/
        bams/
        stringtie/
        trinity_denovo/
        trinity_gg/
        pasa/
```

Trinity (transcript assembler)

Trinity assembles transcript sequences from Illumina RNA-Seq data. It can do de novo assembly or it can do genome-guided. I will do both.

Denovo

Create de novo transcriptome assemblies using Trinity. I used the following script. Also need to load jellyfish (genomescope) and samtools to run trinity. This job needs a long run time. I tried 30 hrs first and that was not nearly enough. Trying 200 hrs second time which worked. You can find the script I ran in my folder. The script is trinity_denovo.sh.

```
Trinity --seqType fq --samples_file samples.txt --max_memory 64G --CPU 8 --output
transcript_data/trinity_denovo
```

Trinity completed de novo assembly of RNA Seq reads. Output file is "trinity_denovo.Trinity.fasta" in transcript_data directory

Genome-Guided Assembly - you will need to map your fastqs to reference genome before you can run trinity genome guided mode.

STAR

Use star to map RNAseq fastqs to genome and produce BAM files. The BAM files created from mapping RNA seq reads to the reference genome with STAR are then used by stringtie and trinity to build transcript assemblies. You can find my scripts star_index.sh and star_map.sh in my folder. The commands are also included here.

At the same time that you are assembling your fastqs with Trinity denovo. You can use STAR to create an index for your reference genome using this command:

```
STAR --runThreadN 4 --runMode genomeGenerate --genomeDir reference/output.star
--genomeFastaFiles reference/your_reference.fa
```

Now we will align all the fastqs to the reference using STAR.

For each sample, map to the reference. For one sample $S:

```
STAR --runThreadN 16 --genomeDir ./reference/${ref}.star \
   --readFilesIn ./transcript_data/fastqs/${S}.r1.fq.gz \
./transcript_data/fastqs/${S}.r2.fq.gz \
    --outFileNamePrefix transcript_data/bams/${S}_ \
    --outSAMtype BAM SortedByCoordinate
```

Next we need to merge all the bams using samtools merge command. This results in an output file called that I called merged.bam. This is now stored in ./transcript_data/. The merged bam file is also what we will use for Stringtie.

But first, let's run the genome-guided trinity.

Taking the bam output from STAR, we use these as input for Trinity's genome-guided mapping command:

```
Trinity --genome_guided_bam merged_sorted_bam_file.bam \
  --max_memory 64G \
  --CPU 8 \
  --output transcript_data/trinity_gg \
  --genome_guided_max_intron 100000
```

Trinity_GG.fasta file stored in./transcript_data/trinity_gg.

Stringtie

assembles and quantifies transcript isoforms from RNA-Seq data. It is compatible with multiple aligners including STAR. It takes the aligned read data as input and processes it to perform transcript assembly and quantification. Stringtie will use the merged.bam file generated by STAR to perform the following step.

Script to run Stringtie:

```
stringtie merged.bam -o transcript_data/stringtie/stringtie.gtf -p 16  -l STRG -f 0.10 -A -c 3 -j 3
```

Stringtie outputs two files (if you include the -A option).

1. The first file is your GTF (genome feature file)
2. The second is a text file that reports of coverage, FPKM, and TPM for each "gene".
   - Coverage is the number of reads supporting each gene call
   - FPKM (Fragments Per Kilobase Million) represents the expression level of a gene or transcript. It is a measure of gene expression normalized by the length of the gene and the total number of reads.
   - TPM (Transcripts Per Million) is also a measure of gene or transcript expression, but it uses a different normalization method. In TPM normalization, the expression level of each transcript is divided by its effective length (accounting for read distribution along the transcript) and then multiplied by a scaling factor to convert the values to the number of transcripts per million in the library.

Output - The stringtie.gtf file you have created will be used later with the PASA software. PASA section below.

PASA

PASA stands for "Program to Assemble Spliced Alignments." It is a bioinformatics software package used for genome annotation and transcriptome analysis. PASA is designed to integrate and improve gene structure annotations by leveraging RNA-Seq data and existing gene predictions.

Dependencies:

Pasa requires the relational database mySQL or SQLite to be installed in order to store and manage its data (think big blast searches). It also requires samtools, BioPerl, minimap2, BLAT, and gmap.

Setting up data for Pasa run:

1. Run 'accession_extract_Pasa.sh' to get de novo transcript accessions. You will need this file to run pasa (step 5).

   ```
   perl /apps/software/gcc-12.1.0/pasapipeline/2.5.3/misc_utilities/accession_extractor.pl
   <./your_denovo_trinity_fasta_file.fasta/ > tdn.accs
   ```

2. Cat together your denovo and genome-guided trinity fastas. You will need this file to run pasa (step 5).

   ```
   cat ../trinity_denovo/Trinity.fasta ../trinity_gg/Trinity-GG.fasta > \
   trinity_transcripts.fa
   ```

3. Clean transcripts with seqclean perl file in the PASA directory (see command below). Use the UniVec fasta file from ftp://ftp.ncbi.nih.gov/pub/UniVec/ or get it from my ./scripts folder. **UniVec** is a non-redundant database of sequences commonly attached to cDNA or genomic DNA during the cloning process. It was developed by staff at NCBI. UniVec_Core is a subset of the full UniVec database.UniVec primarily consists of unique segments from a large number of vectors but also includes many linker, adapter, and primer sequences. Redundant sub-sequences have been eliminated from the database to make searches more efficient and to simplify the interpretation of the results.

   Run the seqclean_pasa.sh script:

   ```
   /apps/software/gcc-12.1.0/pasapipeline/2.5.3/bin/seqclean trinity_transcripts.fa -v
   UniVec.txt -c 16
   ```

   This should leave you with a fasta file trimmed of adapters called trinity_transcripts.fa.clean. You will need this file to run pasa (step 5). You can find my files in /transcript_data/pasa/trinity_combined.

4. You will now need to edit the config file supplied with PASA for alignment and assembly, found in $PASA/pasa_conf/pasa.alignAssembly.TEMPLATE.txt. To point it to the correct database. I created the database with the program sqlite which is installed under gcc/12.1.0. Below I describe how to create the database in sqlite. You can name it whatever you want. This is how pasa manages its data.

   Sqlite

   To create a database in Sqlite:

   ```
   module load sqlite/3.42.0
   sqlite3 your.database.name

   #Now a prompt will open that says “Enter .help for usage hints”
   #you need to tell it to create the database with the name you input above
   #this is the command:

   .databases

   #now you need to exit mysqlite by typing the following:
   .exit
   ```

   Now if you look in your current working directory, you should see your.database.name. Now open the pasa.alignAssembly.config file and edit the DATABASE= to point to your database name.

5. You are now ready to run PASA! With the following script:

   ```
   #!/bin/bash

   #SBATCH --job-name=pasa_run
   #SBATCH --nodes=1
   #SBATCH --ntasks-per-node=1
   #SBATCH --time=24:00:00
   #SBATCH --cpus-per-task=8
   #SBATCH --mem=64gb
   #SBATCH
   --output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse
   _v1/transcript_data/pasa_gmap.out
   #SBATCH
   --error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_
   v1/transcript_data/pasa_gmap.err
   ```

```
module load gcc/12.1.0
module load gmap/2017.11.15
module load minimap2/2.26
module load samtools/1.17
module load openssl/3.1.0
module load sqlite/3.42.0
module load blat/35.1.0
Module load pasapipeline/2.5.3

cd
/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1/tran
script_data

perl /apps/software/gcc-12.1.0/pasapipeline/2.5.3/Launch_PASA_pipeline.pl --config
pasa.alignAssembly.config \
    --ALIGNERS blat,gmap,minimap2 \
    --MAX_INTRON_LENGTH 100000 \
    --trans_gtf ./stringtie/stringtie_c3_j3.gtf \
    --create --run \
    --ALT_SPLICE \
    --CPU 8 \
    --stringent_alignment_overlap 30.0 \
    -T \
    -u trinity_transcripts.fa \
    --genome ../reference/Megaselia_abdita.repeatmasked.fa \
    --transcripts ./trinity_transcripts.fa.clean \
    --TDN tdn.accs
```

Output: This script should give you many files including:

```
<your_sqlite_db_name>.assemblies.fasta
<your_sqlite_db_name>.pasa_assemblies.bed/gff3/gtf
<your_sqlite_db_name>.valid_gmap/blat/minimap2_alignments.bed/gff3/gtf
<your_sqlite_db_name>.failed_gmap/blat/minimap2_alignments.bed/gff3/gtf
<your_sqlite_db_name>.polyAsites.fasta
<your_sqlite_db_name>.alt_splicing_supporting_evidence.txt
  blat/gmap/minimap2.spliced_alignments.gff3
```

VERY IMPORTANT NOTE: Do not delete your sqlite database or alter it. You will need it to update the evidence modeler gff3 file to add UTRs later.

Transdecoder - predicts candidate coding regions from transcript assemblies

TransDecoder identifies candidate coding regions within transcript sequences, such as those generated by de novo RNA-Seq transcript assembly using Trinity, or constructed based on RNA-Seq alignments to the genome using StringTie.

First, run Transdecoder on the pasa assemblies to create training models. Important note: You will use the pasa_asmbls_to_training_set.dbi which can be found in the pasapipeline module (apps/software/gcc-12.1.0/pasapipeline/2.5.3/scripts/pasa_asmbls_to_training_set.dbi) installed on the cluster. HOWEVER, you will need to change the transcoder directory in the script to point to the transcoder software that is installed separately from pasapipline. DO NOT USE the transcoder plugin. It will not work. You can also copy the pasa_asmbls_to_training_set.dbi from /Ayse/Annotation/scripts/ folder which is already edited. Then copy to your directory, load pasapipeline module, and run:

```
module load gcc/12.1.0
module load pasapipeline/2.5.3
module load perl/5.36.0

perl ./pasa_asmbls_to_training_set.dbi  --pasa_transcripts_fasta ${db}.assemblies.fasta
--pasa_transcripts_gff3 ${db}.pasa_assemblies.gff3
```

This script is called RunTransDecoder.sh. You can find it in Ayse's scripts folder.

Output: **${db}.assemblies.fasta.transdecoder.genome.gff3** among other files. You will later merge this file with the braker.gff file to make the predictions.gff3 file for evidence modeler.

Mapping Protein Data (this can be done in parallel to transcriptomes)

Getting the protein data

Download the latest SwissProt database (big file, >80GB). You can also download all entries matching certain criteria from NCBI. I have also used all RefSeq diptera proteins (at https://www.ncbi.nlm.nih.gov/protein search (diptera AND srcdb_refseq[PROP]) AND "flies"[porgn:__txid7147] AND srcdb_refseq[PROP]). You can run this on the cluster in parallel to all the above computations (i.e. you don't need to wait for the transcriptome gffs to do this step).

Exonerate - Exonerate is a bioinformatics tool that performs sequence comparison and alignment tasks. It is commonly used for genomic and protein sequence analysis and is part of the GeneWise and Exonerate suite of programs. Exonerate is designed to efficiently align and compare DNA, RNA, and protein sequences against a reference genome or database. Here we will use it to align proteins from uniprot and refseq databases to our reference genome. You then run exonerate command as seen below for both fasta files (uniprot and refseq).

1. For a single chunk, number N. Exonerate will chunk the search for you if you include the –querychunkid and –querychunktotal. The ID you will need to provide as a variable as 1 - N and then tell exonertate using the –querychunktotal option how many chunks, aka N. I did 250 chunks for uniprot_sprot.fasta and that finished in under 24hrs.

```
exonerate --model p2g \
  --showvulgar no \
  --showalignment no \
  --showquerygff no \
  --showtargetgff yes \
  --maxintron 350000 \
  --minintron 20 \
  --percent 90 \
  --query protein_data/fastas/uniprot_sprot.fa \
  --target reference/reference_genome.fa \
  --querytype protein \
  --targettype dna \
  --querychunkid $S \
  --querychunktotal N \
  --refine region \
  --refineboundary 5000 \
  --softmasktarget yes > \
  protein_data/uniprot_results/uniprot.${S}.gtf
```

Submit by looping over chunks:
for i in $myvar;

```
 do
   sbatch --export=S=${i} RunExonerate.sh
   sleep 1  # Optional: sleep for 1 second between job submissions
done
```

2. Now we need to convert from exonerate's weird format to something that EVidence Modeler (EVM) program will accept. EVM has a perl script you can download called exonerate_gff_to_alignment_gff3.pl. Run with a loop.

```
for i in $myvar
do perl exonerate_gff_to_alignment_gff3.pl ${i} > ${i}.gff3
done
```

3. Remove all empty files.

```
find protein_data/uniprot_results/ -size -500c -delete
```

4. Concatenate all the gff and gtf files (separately). We will need both exonerate gtf format and the EVM gff3.

```
cat exonerate.*.gff3 > uniprot_evm.gff3.intermediate
cat exonerate.*.gtf > uniprot_exonerate.gtf.intermediate
```

5. Remove all blank lines using awk. Remove

```
awk 'NF' uniprot_exonerate.gtf.intermediate >uniprot_exonerate.gtf
awk 'NF' uniprot_evm.gff3.intermediate> uniprot_evm.gff3
```

Do this process again for your refseq file. Refseq file takes much longer than uniprot fasta. I upped the number of hours to 50. This will leave you with 250 exonerate gff files.

Create consensus gff with Evidence Modeler

1. Run Evidence Modeler using all supporting data. Almost all of this is straight from the EVM documentation. Ensure that each source has the correct "source" field ($2) that matches the evm.weights file!! Here is an example of the evm.weights file:

```
ABINITIO_PREDICTION	Augustus	3
ABINITIO_PREDICTION	gmst	3
ABINITIO_PREDICTION	genemark.hmm3	3
PROTEIN	exonerate	5
TRANSCRIPT	assembler-pasa_db.sqlite	10
OTHER_PREDICTION	transdecoder	5
```

2. Make input files. Start in evm/. These are the files you will need to make your predictions.gff, transcripts.gff and proteins.gff files.

Files you need:

- braker.gff3
- reseq_exonerate.gff3
- uniprot_exonerate.gff3
- ${db}.pasa_assemblies.gff3
- ${db}.assemblies.fasta.transdecoder.genome.gff3

```
cat braker.gff Pasa_sqlite.db.assemblies.fasta.transdecoder.genome.gff3 > predictions.gff3

cat refseq_exonerate.gff3 uniprot_exonerate.gff3 > proteins.gff3

cp Pasa_sqlite.db.pasa_assemblies.gff3 transcripts.gff3
```

Koray's comment: uniprot data had to be cleaned because the metadata was excessively long. I used awk to do it.

3. Split up into submission scripts using EVM script. See my script called "partition_evm_inputs.sh." You will need to submit this as a job, it take ~3-5 min to run.

```
perl /apps/software/gcc-12.1.0/evidencemodeler/2.1.0/EvmUtils/partition_EVM_inputs.pl
 --partition_dir Partitions \
--genome ../reference/${ref}.fa \
--gene_predictions predictions.gff3 --protein_alignments proteins.gff3 \
--transcript_alignments transcripts.gff3 \
--segmentSize 3000000 --overlapSize 300000 \
```

```
--partition_listing partitions_list.out
```

4. Now we will run a EVM script to create a list of commands which we will submit individually later. See my script “write_evm_commands.sh” to run as a job.

```
perl /apps/software/gcc-12.1.0/evidencemodeler/2.1.0/EvmUtils/write_EVM_commands.pl
--genome ../reference/${ref}.fasta \
--weights `pwd`/evm.weights \
--gene_predictions predictions.gff3 \
--protein_alignments proteins.gff3 \
--transcript_alignments transcripts.gff3 \
--output_file_name evm.out  --partitions partitions_list.out > \
commands
```

5. This is a script to make submission scripts for each partition job. Run.

```
bash ../scripts/PrepareEvmSubs.sh
```

6. Now we have one script per command. Make a list of all the submission scripts called files and use a loop to submit.

```
ls ./scripts/* > files
myvar=$(cat files)

for i in $myvar
do sbatch ${i};
sleep 1;
done
```

7. After all jobs have completed, recombine all outputs into a single gff3 and convert to gff3 format.

```
perl
/apps/software/gcc-12.1.0/evidencemodeler/2.1.0/EvmUtils/recombine_EVM_partial_outputs.pl
--partitions partitions_list.out --output_file_name evm.out

/apps/software/gcc-12.1.0/evidencemodeler/2.1.0/EvmUtils/convert_EVM_outputs_to_GFF3.pl
--partitions partitions_list.out --output evm.out --genome ../reference/${ref}.fasta

find . -name "evm.out.gff3" -exec cat {} + > EVM.all.gff3
```

8. Remove all blank lines:

```
awk ‘NF’ EVM.all.gff3 > EVM.all.removed.gff3
```

9. Use gff3sort to properly sort the output file. (https://github.com/billzt/gff3sort)

Sort:

```
gff3sort.pl --chr_order natural --precise EVM.all.removed.gff3 > EVM.all.sort.gff3
```

Update Gene Models with PASA

Now we can update the gene models using the PASA assemblies. This is required to identify alternative transcripts and UTRs. This requires at least two rounds because as you improve the assemblies, it will often incorporate more transcripts. Start back in transcript_data/pasa. Ensure you've modified both pasa.annotCompare.config and pasa.alignAssembly.config to point to your **original PASA database** and that the files have all the settings you'd like to use (for the the annotation config file I used all the defaults). You will need to load blat and fasta36 for this step. Important note: pasa expects fasta36 to be named fasta (no idea why). To get around this, I create a symbolic link to fasta36 and name it fasta. The code to do this is included in the slurm script and below. You can get this scripts from my folder ./scripts/RunPasa_update_gene_models.sh

1. Round 1 update. Create a bin/ folder in your working directory before you run this script. You need it for the symbolic link to be create and fasta36 to work.

```
#!/bin/bash

#SBATCH --job-name=pasa_update_3
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=100:00:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=32gb
#SBATCH
--output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1/
transcript_data/pasa/pasa_update.out
#SBATCH
--error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1/tr
anscript_data/pasa/pasa_update.err

module load gcc/12.1.0
module load sqlite/3.42.0
module load pasapipeline/2.5.3
module load samtools/1.18
module load blat/35.1.0
module load fasta36/36.3.8i

cd
/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1/transcript
_data/pasa/
```

```
#Create a symbolic link
if [ ! -e "./bin/fasta" ]; then
   ln -s $(which fasta36) ./bin/fasta
fi

if ! grep -q "export PATH=./bin:\$PATH" ~/.bashrc; then
   echo 'export PATH=./bin:$PATH' >> ~/.bashrc
fi
export PATH=./bin:$PATH

perl
/apps/software/gcc-12.1.0/pasapipeline/2.5.3/scripts/Load_Current_Gene_Annotations.dbi
-c pasa.alignAssembly.config -g ../../reference/Megaselia_abdita.repeatmasked.fa -P
../../EVM/EVM.all.sort.gff3

perl /apps/software/gcc-12.1.0/pasapipeline/2.5.3/Launch_PASA_pipeline.pl -c
pasa.annotCompare.config -A -g ../../reference/Megaselia_abdita.repeatmasked.fa -t
./trinity_combined/trinity_transcripts.fa.clean --CPU 8
```

Note: You can only use --CPU with the Launch_PASA_pipeline.pl. If add multiple threads to the Load_Current_Gene_Annotations.dbi line, it will run and fail. You will get an error in the error file that says “- unknown option” and empty output files.

Ouput: At the end, you’ll have another ${db}.gene_structures_post_PASA_updates.${pid}.gff3 and .bed file

2. Round 2 update. The updated files will be named something like ${db}.gene_structures_post_PASA_updates.${pid}.gff3, where pid is the process id of the annotation update job. After the first update, run another round of updating, but load the annotations from the the new ${db}.gene_structures_post_PASA_updates.${pid}.gff3 file with the -P option (not the EVM gff3).

Ouput: At the end, you’ll have another ${db}.gene_structures_post_PASA_updates.${pid}.gff3, different pid from the first.

3. Remove and blank lines and sort file:

```
awk ‘NF’ ${db}.gene_structures_post_PASA_updates.${pid}.gff3 >
${db}.gene_structures_post_PASA_updates.${pid}.removed. gff3

gff3sort.pl --precise ${db}.gene_structures_post_PASA_updates.${pid}.removed.gff3 >
${db}.gene_structures_post_PASA_updates.${pid}.sort.gff3 and copy to
${ref}.original.gff3
```

Removing TEs using RepeatMasker (Dfam-based) output and Funannotate's repeats.dmnd database

Go to your repeatmasker output folder and find the .out file. This is what you need for the next step.

To get the repeatmasker elements into a bed file format:

1. Convert repeatmasker out file to gff3

   ```
   /apps/software/gcc-12.1.0/repeatmasker/4.1.5/util/rmOutToGFF3.pl
   Mab_ref_genome.fasta.out > Mab_ref_genome.fasta.gff3
   ```

2. Then convert gff3 to a bed file

   ```
   Awk '{print $1 "\t" $4 "\t" $5 }' Mab_ref_genome.fasta.gff3 >
   Mab_ref_genome.repeatmasker.bed
   ```

To get a fasta file of annotated proteins:

1. Find the cufflinks_gffread.sh script. The gffread command can take your genome fasta file and the gff3 (from EVM and updated with PASA) and generate a proteins.fa file.

   ```
   module load gcc/12.1.0
   module loaf cufflinks/

   gffread -y your.ouput.proteins.fa -g your.genome.fa your.annotation.gff3
   ```

2. Check your out.proteins.fa file for periods in the protein sequence. If present, diamond will not be able to parse the file (https://github.com/nextgenusfs/funannotate/issues/459 gives a description of the issue). To get around this, remove all periods from the protein sequences with this command:

   ```
   awk '/^>/ {if (seq) print seq; print; seq=""} !/^>/ {gsub(/\./, "", $0); seq = seq $0} END {print
   seq}' inputfile.fasta > outputfile.fasta
   ```

3. Find any gene models that match (90%) with your repeat bed file generated with Repeat Masker above. Uses python script called Find_bed_file_repeats.py and a job script to run the python script called Run_Find_bed_files_repeats.py.

   This use a funannotate command called RemoveBadModels. If you specify "overlap" then the function will perform bedtools intersect on your bed file and gff3 file. The output will be called genome.repeats.to.remove.gff3 (you do not have to define it in the script). You can use script called **Find_bed_file_repeats.py**:

   ```
   #!/usr/bin/env python
   # -*- coding: utf-8 -*-
   from __future__ import (absolute_import, division,
   ```

```
                    print_function, unicode_literals)
import sys
import os
import subprocess
import shutil
import argparse
import json
import datetime
import funannotate.library as lib
from natsort import natsorted

lib.setupLogging('Find_bed_repeats.log')

# lazy. just modify here.
Blast_rep_remove = os.path.join('repeat.dmnd.blast.txt')
EVM_out = os.path.join('Megaselia_abdita.original.gff3')
EVMCleanGFF=os.path.join('Megaselia_abdita.clean.gff3')
EVM_proteins= os.path.join('Megaselia_abdita.original.proteins.fa')
RepeatMasker = os.path.join('Mab_ref_genome.repeatmasker.bed')
repeat_filter = ['overlap']
min_protlen=50
FUNDB=os.path.join('/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/p
y_2/lib/python2.7/site-packages/funannotate/')

lib.RemoveBadModels(EVM_proteins, EVM_out, min_protlen, RepeatMasker,
Blast_rep_remove, os.path.join('.'), repeat_filter, EVMCleanGFF)

total = lib.countGFFgenes(EVMCleanGFF)
lib.log.info("{:,} gene models remaining".format(total))
```

To run this script you will need to use the conda environment in which you installed funannotate. See my script Run_Find_bed_file_repeats.sh:

```
#!/bin/bash

#SBATCH --job-name=count_genes
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=05:00:00
#SBATCH --cpus-per-task=1
#SBATCH --mem=16gb
#SBATCH
--output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Cal_Annotation_v1/c
ount.out
```

```
#SBATCH
--error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Cal_Annotation_v1/count.err

module purge
module load gcc/12.1.0
module load miniconda3/23.1.0

source /apps/software/gcc-12.1.0/miniconda3/23.1.0/etc/profile.d/conda.sh
conda activate /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/py_2
export
FUNANNOTATE_DB=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/py_2/lib/python2.7/site-packages/funannotate

cd /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Cal_Annotation_v1

python Find_bed_file_repeats.py

conda deactivate
```

Now we need to removing the Repeat Masker output (your bed file) based repeats/TEs from gff3 file. Use the Remove_bed_TEs.sh but change the gff3_file to your genome.repeats.to.remove.gff file and run.

```
#!/bin/bash/

#input your file paths here
gff_file="./Megaselia_abdita.bed_repeats_removed.gff3"
list_file="./genome.repeats.to.remove.gff"
output_file="./Megaselia_abdita.bed_repeats_removed.gff3"

#Use grep to exclude lines from the gff file if they match exacty with line in the list file
grep -vFf "$list_file" "$gff_file" > "$output_file"

echo "Processing complete"
```

To run script:

```
Bash Remove_bed_repeat.sh
```

This removes all lines in the gff file that match the genome.repeats.to.remove.gff file.

Now all of the overlapping regions from your repeatmasker output are removed and we can move on to looking for TEs with a funannotate function (uses BLAST) and their database.

4. Blast your gene models (gff3 file) against known TE database (funannotate)

   This also uses the funannotate command called RepeatBlast. To blast your gene models against the repeats (TE) database of funannotate see my script called **Find_blast_repeats.py.** You will need to run this script with the Run_Find_blast_repeats.sh script. See below.

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from __future__ import (absolute_import, division,
                        print_function, unicode_literals)

import sys
import os
import subprocess
import shutil
import argparse
import json
import datetime
import funannotate.library as lib
from natsort import natsorted

lib.setupLogging('blast_db.log')

#modify to your file names here
EVM_out = os.path.join('Megaselia_abdita.bed_repeats_removed.gff3')
EVM_proteins= os.path.join('Megaselia_abdita.bed_repeats_removed.proteins.fa')
cpus=1
evalue=1e-10
FUNDB=os.path.join('/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/p
y_2/lib/python2.7/site-packages/funannotate/')

#output
Blast_rep_remove = os.path.join('repeat.dmnd.blast.txt')

#Here we are blast against repeats.dmnd and output the repeat.dmnd.blast.txt file
lib.RepeatBlast(EVM_proteins, cpus, 1e-10, FUNDB, os.path.join('.'), Blast_rep_remove)
```

   Now run Run_Find_blast_repeats.sh:

This will **output** a file called **repeat.dmnd.blast.txt** which contains all the TEs that are in the funannotate library and in your gff file. These will need to be removed. But they are not in a gff3 format for whatever reason (sorry!) so we will need to do a bit of reformatting from the original gff3 to get all the lines that need to be removed. Use the script **Reformat_blast_TE_list.sh** but make sure to change the files paths to match your file names and use in a job, takes a while to run this:

```
#!/bin/bash

# Define file paths
gff_file="./Megaselia_abdita.original.gff3"
dmnd_blast_file="./repeat.dmnd.blast.txt"
output_file="./Megaselia_abdita.blast_repeats_removed.final.gff3"

# Step 1: Extract IDs from the blast file
awk '{print $1}' "$dmnd_blast_file" > Blast_TE_repeats.txt

# Prepare a temporary and final output file
temp_file="./temp_gff_file.gff"
cp "$gff_file" "$temp_file"

# Function to filter GFF file based on IDs with a specified prefix in the attribute (Parent= or
ID=)
filter_gff() {
    local prefix=$1
    local file_list=$2
    local temp_out="./temp_out.gff"

    while read -r id; do
        grep -vE "${prefix}${id}([; ].*|$)" "$temp_file" > "$temp_out"
        mv "$temp_out" "$temp_file"
    done < "$file_list"
}

# Step 2: Filter based on "Parent=" attribute
filter_gff "Parent=" Blast_TE_repeats.txt

# Step 3: Filter based on "ID=" attribute (reusing the list)
filter_gff "ID=" Blast_TE_repeats.txt

# Step 4: Modify the IDs in the list
sed -i 's/evm.model/evm.TU/g' Blast_TE_repeats.txt

# Step 5: Filter based on modified "ID=" attribute
```

```
filter_gff "ID=" Blast_TE_repeats.txt

# Move the final temp file to the output
mv "$temp_file" "$output_file"

echo "Filtering complete. Output is in $output_file"
```

Make your TE-free protein fasta file

1. Now use gffread from cufflinks to get a new fasta file of your protein sequences:

```
module load gcc/12.1.0
module load cufflinks/2.2.1

gffread Megaslia_abdita.All_repeats_removed.gff3 -g
Megaselia_abdita.repeatmasked.fa -y
Megaselia_abdita.All_repeats_removed.proteins.fa
```

2. And use your awk statement to remove periods (diamond does not work otherwise and downstream softwares work with diamond):

```
awk '/^>/ {if (seq) print seq; print; seq=""} !/^>/ {gsub(/\./, "", $0); seq = seq $0} END
{print seq}' inputfile.fasta > outputfile.fasta
```

Run BUSCO on protein sequences

BUSCO stands for "Benchmarking Universal Single-Copy Orthologs." It is a tool used in genomics for assessing genome assembly and annotation completeness with single-copy orthologs. Here we will run the busco software on our genome annotation using several different lineages/databases.

The -i option is for your proteins fasta. -o is the name of the output folder. -l is the lineage/database you want to compare your proteins to, in this case eukaryota. You should also run metazoa, insecta, arthropoda, and diptera. The -m option indicates the mode you want to run busco in: protein (prot), genome, or transcriptome. -c option is for the number of cpus. You can find this script as BUSCO.sh in my scripts folder.

```
#!/bin/bash

#SBATCH --job-name=busco
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=00:10:00
#SBATCH --cpus-per-task=8
#SBATCH --mem=16gb
#SBATCH
  --output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v
  1/busco_eukaryota.out
#SBATCH
  --error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1/
  busco_eukaryota.err

cd /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Annotation_Ayse_v1

module load gcc/12.1.0
module load python/3.10.5

busco -i ./Megaselia_abdita.repeats_removed.proteins.fa -o busco_output_eukaryota -l
  eukaryota_odb10 -m prot -c 8
```

Get Genes Names with EggNOG-mapper

EggNOG-mapper will functionally annotate your genome by assigning gene names and other annotations based orthologous sequences.

1. Choose Taxonomic Scope to run in HMMER mode.

2. Download eggnog databases in interactive mode (can't run in a job because it requires user input)
   a. To ask for an interactive job run this:
   ```
   srun --pty --nodes=1 --ntasks-per-node=1 --cpus-per-task=1 --mem=8G bash
   ```

   b. To download the databases (diamond, pfam, hmmer):
   ```
   module load gcc/12.1.0
   module load eggnog-mapper/2.1.12

   download_eggnog_data.py -P -M -H -d 50557 --dbname Insecta --data_dir
       ./eggnog_data

   download_eggnog_data.py -P -M -H -d 7147 --dbname Diptera --data_dir ./eggnog_data
   ```

3. Run EggNOG-mapper:
   Execute eggNOG-mapper with appropriate arguments and tell it where the eggnog data can be found with the --data_dir option. You will want to run in HMMER mode which is much slower than diamond but gives many more annotations. Give this a long time to run and a good number of cpus. My script is called eggnog_mapper.sh. This is an example script:

   ```
   module load gcc/12.1.0
   module load eggnog-mapper/2.1.12

   emapper.py -m hmmer -i Megaselia_abdita.All_repeats_removed.proteins.fa -o
   output_prefix -d Diptera --data_dir ./eggnog_data --cpu 24 --decorate_gff
   ./Megaselia_abdita.All_repeats_removed.gff3 --report_orthologs --excel
   ```

4. Review and Analyze the Output:
   Once eggNOG-mapper completes, you will have several output files with varying levels of detail. The `.annotations` file will provide you with functional annotations, including gene names, for each of your input sequences. The annotated gff will be output_prefix.emapper.decorated.gff. See next note, there is a little more to do. The decorated file is not as useful as I thought….

****eggnog only annotates the transcripts which is a problem downstream**. Specifically for converting to a gtf format. To get around this use the script AddEggnogToGff3.pl to replace the NAME= of each gene (not transcript) in column 9 of the gff to the annotation provided by eggnog.

To do this you will need your repeats remove gff file, the emapper.annotations file, and a file you will make called txt2gene.txt.

To make the tx2gene file. This stands for transcript to gene. So you want the transcript ID (ID=) in the first column and the respective parent ID (Parent=) in the second column for everything except the gene. Essentially copy the repeats remove gff, remove all lines with gene in the 3rd column and pull the string following ID= and place in the first column of a new file, then pull the string following Parent= from the same line and place in column two. This script is called make_txt2gene_file.sh:

```
awk -F'\t' '$3 != "gene" {
        split($9, a, ";");
        for (i in a) {
          if (a[i] ~ /^ID=/) {
            split(a[i], id, "=");
            printf "%s\t", id[2];
          } else if (a[i] ~ /^Parent=/) {
            split(a[i], parent, "=");
            printf "%s\n", parent[2];
          }
        }
      }' your_file.gff > output_file.tsv
```

Now run the run_AddEggnogToGff3.pl script:

```
#!/bin/bash
#SBATCH --job-name=run_perl
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=00:05:00
#SBATCH --cpus-per-task=1
#SBATCH --mem=8gb
#SBATCH --output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/perl.out
#SBATCH --error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/perl.err

module load gcc/12.1.0
module load perl/5.36.0

cd /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse
```

```
perl AddEggnogToGff3_more_descriptive.pl --gff3
Megaselia_abdita.All_repeats_removed.gff3 --eggNOG emapper.annotation --gff3-out
Megaselia_abdita.annotated.gff3 --table-out table.out --tx2gene tx2gene.txt
```

FYI about eggnog gene names

*Unfortunately, eggnog uses the consensus name of the gene orthologs. Preferred gene names are the consensus of names among many orthologs. Those names are, in most cases HUGO names, but they could also come from other sources. Sequence names in eggnog are not always UniprotKB, there are also Ensembl, RefSeq, etc. So even though in diptera the gene name is e (for ebony), it will be annotated as ACSF (human/mammal ortholog). Some genes that are specific to insects or diptera will have the flybase name, but that is because there isn't an ortholog. So zen and bicoid are annotated with those names.

Check Annotation Statistics with [agat]

If you want to run agat with a job script (not interactive) you will need to add the following line to your script:

```
source /apps/software/gcc-12.1.0/miniconda3/23.1.0/etc/profile.d/conda.sh
```

1. agat is not installed on Randi. You can install it in the py_2 conda environment.

```
module load gcc/12.1.0
module load miniconda3/.23.1.0

conda activate /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/py_2

conda install -c bioconda agat
```

2. After successful installation. Run like so:

```
conda activate /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/py_2
source /apps/software/gcc-12.1.0/miniconda3/23.1.0/etc/profile.d/conda.sh

agat_sp_statistics.pl --gff test_diptera.emapper.decorated.gff -f
../Megaselia_abdita.repeatmasked.fa -d --output ./agat_output

conda deactivate
```

3. If you need to convert your gtf to a gff3 file:

```
agat_convert_sp_gxf2gxf.pl -g test.gtf -o test.gff3
```

Ouput: Your output should be agat_ouput.

Uploading to NCBI (annotated genomes)

Transferring the ‘name’ attribute of the mRNA feature to all of the mRNA feature’s children CDS features ‘product’ attribute is necessary if uploading to NCBI and you want to avoid every CDS being annotated as just ‘hypothetical protein’

I apologize to anyone who is about to do this. I was stressed and needed to get these names attached to the CDS features quickly. So This will involve R and bash.

R script:

```
library(rtracklayer)

# Set the working directory
setwd("C:/Users/ayset/Downloads/")

# Load the GFF file
gff_file <- "Clogmia.gff3"
gff <- import(gff_file)

# Step 1: Extract 'mRNA' features and map their 'Name' attributes to 'ID'
mRNA_features <- gff[gff$type == "mRNA"]
mRNA_name_map <- mcols(mRNA_features)[, c("ID", "Name")]

# Step 2: Extract 'CDS' features
cds_features <- gff[gff$type == "CDS"]

# Step 3: Propagate 'Name' to 'product' in CDS features
cds_features$product <- sapply(mcols(cds_features)$Parent, function(parent_id) {
  matching_name <- mRNA_name_map[mRNA_name_map$ID == parent_id, "Name"]
  if (length(matching_name) > 0) return(matching_name) else return(NA)
})

# Step 4: Add the 'product' column to the original GFF object if it doesn't exist
if (!"product" %in% colnames(mcols(gff))) {
  mcols(gff)$product <- NA_character_
}

# Step 5: Update the `product` attribute for `CDS` features in the GFF object
cds_indices <- which(gff$type == "CDS")
mcols(gff)$product[cds_indices] <- cds_features$product

# Step 6: Export the updated GFF without altering its original order
export(gff, "Clogmia_3.gff", format = "gff3")
```

Moving back to text languages now:

1. Use this command to remove the Note attribute from the gene features (in Clogmia and Megaselia these were some nonsense numbers)

   ```
   awk -F'\t' '{ if ($3 == "gene") { sub(/;Note=[^;]*/, "", $9) } print }' OFS='\t' input.gff > output.gff
   ```

2. Use this command to replace any dashes in the product attribute of the CDS features with putative since that is what the dashes mean (note, you must use 'putative', if you substitute any other word table2asn messes up):

   ```
   awk -F'\t' '{if ($9 ~ /product=-/){sub(/product=-.*/, "product=putative", $9)} print}' OFS='\t' Megaselia.gff3 > out
   ```

Now you are ready to use NCBI's Table2asn software to make a sqn file (only for annotated genomes):

https://www.ncbi.nlm.nih.gov/genbank/genomes_gff/

1. Run table2asn on Randi. First, downloaded table2asn via bioconda to a python environment. (In my case py_env_3) then activate and run the command in a script called script table2asn_conda.sh:

```
#!/bin/bash
#SBATCH --job-name=table2asn
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --time=01:05:00
#SBATCH --cpus-per-task=1
#SBATCH --mem=8gb
#SBATCH --output=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/asn_meg.out
#SBATCH --error=/gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/asn_meg.err

module purge
module load gcc/12.1.0
module load miniconda3/23.1.0

source /apps/software/gcc-12.1.0/miniconda3/23.1.0/etc/profile.d/conda.sh
conda activate /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/Conda/py_env_3/
cd /gpfs/data/schmidtott-lab/Labserver/Personal_Folders/Ayse/
```

```
table2asn -M n -J -c w -euk -t template_Meg.sbt -gaps-min 10 -l proximity-ligation
-locus-tag-prefix ACFI09 -j "[organism=Megaselia abdita][isolate=Sander]" -i
Megaselia_abdita.repeatmasked.fa -f Megaselia_vNCBI_2.gff3 -o MegAbd_output_file.sqn -Z -V
b

conda deactivate
```

NOTES
The **locus-tag-prefix should have been assigned when you made the BioProject Accession for your genome submission.
You have to make the **template.sbt file on NCBI and download it
****proximity-ligation** is the correct evidence for joining of reads with gaps based on any chromatin capture methods (our genomes are not scaffolds based on paired-ends which they suggest using on the website)

Uploading your .sqn file to NCBI Genomes

Now you can transfer your .sqn file to NCBI via ftp from midway.

1. Move your file to midway using scp:

   ```
   scp ./youfile.sqn your_username@midway.rcc.chicago.edu:/project2/uschmid/
   ```

2. Connect to NCBI's server via ftp

   ```
   ftp ftp-private.ncbi.nlm.nih.gov
   ```

3. Use NCBI's provided login (you have to have started a genome submission on NCBI to get to the username and password and you designated folder on NCBI)

4. See instructions on ncbi submission portal for moving your .sqn file to the pre-load folder provided by NCBI